

Monte Carlo Simulation for Stock Price Predictive Modeling

A Pure Python and MySQL Implementation Architecture

1. Introduction to the Monte Carlo Algorithm

The Monte Carlo algorithm is a robust computational methodology that relies on repeated random sampling to obtain deterministic results. Initially conceptualized by scientists Stanislaw Ulam and John von Neumann in the 1940s, the algorithm solves complex probabilistic problems by simulating a massive system of potential trajectories.

In financial analytics, a single historical dataset provides only a single chronological outcome. To model market uncertainty, the Monte Carlo method simulates thousands of alternative future scenarios. By aggregating these distinct randomly generated pathways, quant algorithms can calculate structural risks, predict price distributions, and define confidence intervals for prospective stock values.

2. Mathematical Foundations: Geometric Brownian Motion (GBM)

Stock price behavior cannot be modeled as a pure random walk because prices cannot drop below zero, and assets generally exhibit an upward or downward growth tendency based on fundamental performance. To resolve this, financial mathematics uses **Geometric Brownian Motion (GBM)**.

GBM posits that the continuous-time stochastic process governing stock prices satisfies the following differential relationship:

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

Where:

- S_t is the stock price at time t .
- μ (Drift) represents the constant expected periodic rate of return.
- σ (Volatility) represents the absolute standard deviation of stock returns, capturing market volatility.
- dW_t is a Wiener process increment (Brownian motion), corresponding to a random draw from a normal distribution with a mean of 0 and variance of dt .

By integrating this differential equation via Ito's Lemma, we obtain the discrete time-step operational formula utilized for numeric simulation models:

$$S_{t+\Delta t} = S_t \times \exp((\mu - \sigma^2/2)\Delta t + \sigma Z \sqrt{\Delta t})$$

Where Z represents a standard normal random variable ($Z \sim N(0, 1)$). This formulation splits asset trajectory changes into two distinct parts:

- **Deterministic Drift:** $(\mu - \sigma^2/2)\Delta t$, providing the predictable directional trend of the asset over time.
- **Stochastic Shock:** $\sigma Z \sqrt{\Delta t}$, introducing the unpredictable random market shocks driven by random variables.

Implementation Strategy: Because this architecture explicitly prohibits external mathematical packages like NumPy, standard normal distributions (Z) are generated using Python's built-in `random.gauss(0, 1)` function.

3. System Architecture & Database Design

The pipeline utilizes a MySQL database instance to handle both input historical analytics and output data streams generated by the simulation engine. This ensures relational data persistence, enabling decoupled querying for dashboard generation or probability metrics execution.

Database Schema Setup

Below is the raw relational structural schema designed to store input price components and generated iterations:

```
CREATE DATABASE IF NOT EXISTS market_simulation;
USE market_simulation;

-- Stores historical attributes calculated for baseline drift and volatility
CREATE TABLE IF NOT EXISTS stock_metadata (
    ticker VARCHAR(12) PRIMARY KEY,
    last_close_price DECIMAL(14,4) NOT NULL,
    calculated_drift DECIMAL(8,6) NOT NULL,
    calculated_volatility DECIMAL(8,6) NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Stores raw statistical outputs of each independent Monte Carlo simulation path
CREATE TABLE IF NOT EXISTS simulation_outputs (
    simulation_id INT AUTO_INCREMENT,
    ticker VARCHAR(12),
    run_index INT NOT NULL,
    day_index INT NOT NULL,
    predicted_price DECIMAL(14,4) NOT NULL,
    PRIMARY KEY (simulation_id, run_index, day_index),
    FOREIGN KEY (ticker) REFERENCES stock_metadata(ticker)
);
```

4. Pure Python + MySQL Implementation

The program below uses only `math`, `random`, and `mysql.connector`. It extracts baseline values, simulates 1,000 future asset trajectories over a 30-day forecast horizon, and saves the data directly into the database.

```

import math
import random
import mysql.connector

def get_database_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="your_secure_password",
        database="market_simulation"
    )

def fetch_stock_parameters(ticker):
    conn = get_database_connection()
    cursor = conn.cursor(dictionary=True)

    query = "SELECT last_close_price, calculated_drift, calculated_volatility FROM
stock_metadata WHERE ticker = %s"
    cursor.execute(query, (ticker,))
    result = cursor.fetchone()

    cursor.close()
    conn.close()

    if not result:
        raise ValueError(f"Ticker {ticker} not initialized in stock_metadata table.")
    return result

def run_monte_carlo_simulation(ticker, total_simulations=1000, days_to_predict=30):
    # Fetch parameters from MySQL
    params = fetch_stock_parameters(ticker)
    S0 = float(params['last_close_price'])
    mu = float(params['calculated_drift'])
    sigma = float(params['calculated_volatility'])

    dt = 1.0 # Daily step increment

    # Establish connection for writing outputs
    conn = get_database_connection()
    cursor = conn.cursor()

    # Generate unique global simulation identifier
    cursor.execute("SELECT IFNULL(MAX(simulation_id), 0) + 1 FROM simulation_outputs")
    sim_id = cursor.fetchone()[0]

    print(f"Starting Simulation ID {sim_id} for {ticker} over {days_to_predict} days...")

    insert_query = (
        "INSERT INTO simulation_outputs (simulation_id, ticker, run_index, day_index,
predicted_price) "

```

```

        "VALUES (%s, %s, %s, %s, %s)"
    )

    batch_data = []

    for run in range(1, total_simulations + 1):
        current_price = S0
        for day in range(1, days_to_predict + 1):
            # Pure mathematical generation of standard normal distribution  $Z \sim N(0,1)$ 
            Z = random.gauss(0, 1)

            # Geometric Brownian Motion equation application
            drift_term = (mu - (sigma ** 2) / 2.0) * dt
            shock_term = sigma * Z * math.sqrt(dt)
            current_price = current_price * math.exp(drift_term + shock_term)

            batch_data.append((sim_id, ticker, run, day, current_price))

            # Flush batch tracking arrays to prevent excessive RAM utilization
            if len(batch_data) >= 5000:
                cursor.executemany(insert_query, batch_data)
                conn.commit()
                batch_data = []

    if batch_data:
        cursor.executemany(insert_query, batch_data)
        conn.commit()

    cursor.close()
    conn.close()
    print(f"Simulation {sim_id} successfully persisted to the database.")
    return sim_id

def calculate_target_probabilities(sim_id, target_price):
    conn = get_database_connection()
    cursor = conn.cursor()

    # Query terminal day prices across all generated simulation blocks
    query = """
        SELECT predicted_price
        FROM simulation_outputs SO
        WHERE SO.simulation_id = %s
        AND SO.day_index = (SELECT MAX(day_index) FROM simulation_outputs WHERE
simulation_id = %s)
    """
    cursor.execute(query, (sim_id, sim_id))
    terminal_prices = [row[0] for row in cursor.fetchall()]

    cursor.close()
    conn.close()

```

```
total_runs = len(terminal_prices)
successful_runs = sum(1 for price in terminal_prices if price >= target_price)

probability = (successful_runs / total_runs) * 100 if total_runs > 0 else 0.0
return probability, total_runs
```

5. Analytical Verification & Probability Calculation

Once the generated arrays have been successfully saved into the database, predictive probability queries can be executed to determine structural risk thresholds. For instance, computing the likelihood that a stock will exceed a specific target value at the end of a 30-day window is done by isolating the terminal nodes across all runs.

Metric Metric Descriptor	Relational Query Formula / SQL Implementation	Analytical Value Achieved
Target Probability Likelihood	COUNT(price > target) / TOTAL_RUNS	Computes absolute confidence of positive asset return.
Value at Risk (VaR)	ORDER BY price ASC LIMIT 5% Total	Defines maximum losses under normal market environments.
Expected Value (Mean Projection)	AVG(predicted_price)	Yields the central expected trajectory path.

By structuring the system this way, we eliminate dependencies on memory-heavy, multi-gigabyte data libraries. This allows the simulation logic to run inside extremely resource-constrained servers, edge nodes, or directly within native internal database stored procedures.